Finance Smart

Documentação de Testes Automatizados
com Playwright e C#

Teste de conexão com o banco de dados e do fluxo de recuperação de senha

Repositório: github.com/LucasPiet/Finance-Smart

Sumário

1. O que é o Finance Smart
2. Por que Playwright e por que C#
3. Visão geral da estratégia de testes
4. Preparando o ambiente
5. Estrutura do projeto de testes
6. Arquivo de configuração (TestConfig.cs)
7. Teste de conexão com o banco de dados (DatabaseTests.cs)
8. Ambiente de e-mail para testes (Mailpit)
9. Teste de recuperação de senha (ForgotPasswordTests.cs)
10. Como executar os testes
11. Problemas comuns e soluções
12. Boas práticas e próximos passos
13. Glossário

1. O que é o Finance Smart

O **Finance Smart** é um sistema de gestão financeira que permite ao usuário controlar suas receitas e despesas. Ele foi construído com **arquitetura de microserviços**: em vez de um único programa grande, o sistema é dividido em serviços pequenos e independentes, cada um com uma responsabilidade.

Componente	Função	Porta
Front (React + TypeScript)	Interface web usada pelo usuário	5173
auth_service (FastAPI/Python)	Autenticação de usuários, preparado para JWT, e recuperação de senha	8001
transacoes_service (FastAPI/Python)	Cadastro de receitas e despesas e controle de saldo	8002
email_service (FastAPI/Python)	Envio de e-mails (ex.: recuperação de senha)	8003
sqlserver (SQL Server 2022)	Banco de dados relacional FINANCE_SMART; triggers garantem a consistência do saldo	1434 (host) / 1433 (container)
db-init	Container auxiliar que cria o banco, as tabelas e a trigger de saldo na subida	-

Todos os componentes são executados com **Docker Compose**, o que permite subir o sistema completo com um único comando:

```
docker compose up --build
```

2. Por que Playwright e por que C#

2.1 O que é o Playwright

O **Playwright** é uma ferramenta de automação de testes criada pela Microsoft. Ele controla navegadores reais (Chromium, Firefox e WebKit) e faz o que um usuário faria: abrir páginas, clicar, digitar e verificar o que aparece na tela. Também consegue fazer chamadas HTTP diretas para APIs. Esse tipo de teste é chamado de **teste de ponta a ponta** (end-to-end, ou E2E).

2.2 Por que usar C#, se o backend é em Python

Os testes E2E interagem com o sistema **de fora**, pelo navegador e pelo protocolo HTTP. Por isso, a linguagem dos testes é independente da linguagem do sistema testado. O Playwright só precisa acessar `http://localhost:5173` e as APIs, e não importa se por trás existe Python, Java ou C#.

Foi escolhido **C#** porque é a linguagem utilizada pela empresa, o que facilita a manutenção dos testes por quem já trabalha com .NET. O Playwright tem suporte oficial a C# (pacote `Microsoft.Playwright.NUnit`), com a mesma qualidade dos suportes a TypeScript e Python.

2.3 Por que NUnit

O **NUnit** é um framework de testes tradicional do ecossistema .NET. Ele fornece os atributos `[Test]`, `[SetUp]` e `[Category]`, o executor (`dotnet test`) e os relatórios de resultado. O Playwright oferece integração pronta com ele.

3. Visão geral da estratégia de testes

Foram criados dois grupos de testes, cada um validando uma parte crítica do sistema:

Grupo	O que valida	Ferramenta principal	Por quê
Banco de dados (Category=Database)	Conexão com o SQL Server, existência do banco FINANCE_SMART, das tabelas e da trigger de saldo	NUnit + Microsoft.Data.SqlClient	Se o banco não sobe ou não foi inicializado, todo o sistema falha. É o teste mais básico e rápido.
Recuperação de senha (Category=Email)	Tela Esqueci minha senha, chamada à API de autenticação e chegada do e-mail	Playwright (navegador + API) + Mailpit	Valida o fluxo completo: front, auth_service, email_service e envio, de ponta a ponta.

Observação importante: o Playwright testa navegadores e chamadas HTTP, não bancos de dados. Por isso o teste de conexão usa o driver oficial do SQL Server dentro do mesmo projeto NUnit. É uma prática comum: um único projeto de testes pode combinar várias ferramentas, cada uma na sua função.

4. Preparando o ambiente

4.1 Pré-requisitos

- **.NET SDK** (8 ou superior). Confirme com `dotnet --version`.
- **Docker Desktop** com o Finance Smart rodando.
- **PowerShell**, que já vem no Windows (usado para instalar os navegadores do Playwright).

4.2 Criação do projeto de testes

O projeto é criado a partir do template do NUnit. Os comandos abaixo devem ser executados na pasta onde ficará o projeto (neste caso, `Finance-Smart\Tester`):

```
dotnet new nunit -n FinanceSmart.Tests
cd FinanceSmart.Tests
dotnet add package Microsoft.Playwright.NUnit
dotnet add package Microsoft.Data.SqlClient
dotnet build
powershell -ExecutionPolicy Bypass -File .\bin\Debug\net8.0\playwright.ps1 install
```

Comando	O que faz
dotnet new nunit	Cria um projeto de testes NUnit com a estrutura padrão (arquivo .csproj e um teste de exemplo).
dotnet add package Microsoft.Playwright.NUnit	Adiciona o Playwright já integrado ao NUnit. Fornece as classes PageTest e PlaywrightTest.
dotnet add package Microsoft.Data.SqlClient	Adiciona o driver oficial para conectar ao SQL Server (usado no teste de banco).
dotnet build	Compila o projeto. É nessa etapa que o script playwright.ps1 é gerado.
playwright.ps1 install	Baixa os navegadores (Chromium, Firefox, WebKit) usados nos testes. Só é necessário uma vez.

Sobre a pasta net8.0: o nome da pasta em `bin\Debug` depende da versão do .NET do projeto (por exemplo, net8.0 ou net11.0). Use o nome que existir na sua máquina (confira com `dir bin\Debug`).

4.3 Descobrindo seletores com o gerador de código (codegen)

Para escrever testes de interface é preciso informar ao Playwright *qual elemento* da tela clicar ou preencher. O Playwright possui um **gravador** que observa suas ações no navegador e gera o código C# correspondente:

```
powershell -ExecutionPolicy Bypass -File .\bin\Debug\net8.0\playwright.ps1 codegen http://localhost:5173
```

Basta clicar em Esqueci minha senha, digitar um e-mail e enviar. O código gerado no Inspector é copiado para o teste. Isso evita erros de digitação em textos de botões e labels.

5. Estrutura do projeto de testes

```
FinanceSmart.Tests/
|-- FinanceSmart.Tests.csproj     (pacotes e versão do .NET)
|-- TestConfig.cs                 (URLs, portas e string de conexão)
`-- Tests/
    |-- DatabaseTests.cs          (testes de banco de dados)
    `-- ForgotPasswordTests.cs    (testes do fluxo de recuperação de senha)
```

Por que separar assim? A configuração fica em um único lugar (se uma porta mudar, altera-se um arquivo só) e cada arquivo de teste cobre uma funcionalidade, o que facilita encontrar e manter os testes.

6. Arquivo de configuração (TestConfig.cs)

```
namespace FinanceSmart.Tests;

public static class TestConfig
{
    public static string FrontUrl   => Env("FRONT_URL", "http://localhost:5173");
    public static string AuthUrl    => Env("AUTH_URL", "http://localhost:8001");
    public static string MailpitUrl => Env("MAILPIT_URL", "http://localhost:8025");

    // Porta 1434 = mapeamento do docker-compose (1434:1433)
    public static string DbConnection => Env("DB_CONNECTION",
        "Server=localhost,1434;Database=FINANCE_SMART;User Id=sa;Password=Smart123;" +
        "TrustServerCertificate=True;Connect Timeout=10");

    private static string Env(string key, string fallback) =>
        Environment.GetEnvironmentVariable(key) ?? fallback;
}
```

Como funciona

- **Classe estática:** não precisa ser instanciada; qualquer teste acessa `TestConfig.FrontUrl` diretamente.
- **Método Env:** procura uma variável de ambiente; se ela não existir, usa o valor padrão (local). Assim os mesmos testes rodam na sua máquina e em outro ambiente (como um servidor de CI) sem alterar código, apenas definindo variáveis.
- **Porta 1434:** no `docker-compose.yml` o SQL Server expõe `1434:1433`, ou seja, a porta 1433 do container é acessada pela 1434 da máquina. Por isso a string de conexão usa 1434.
- **TrustServerCertificate=True:** o SQL Server do Docker usa um certificado autoassinado; essa opção evita erro de certificado em ambiente de teste local.
- **Connect Timeout=10:** o teste falha em até 10 segundos se o banco estiver fora do ar, em vez de ficar travado.

> **Segurança:** a senha Smart123 é a mesma do `docker-compose.yml` e serve apenas para ambiente local. Em um projeto real, credenciais não devem ficar no código: devem vir de variáveis de ambiente ou de um cofre de segredos.

7. Teste de conexão com o banco de dados (DatabaseTests.cs)

```
using Microsoft.Data.SqlClient;

namespace FinanceSmart.Tests.Tests;

[TestFixture]
[Category("Database")]
public class DatabaseTests
{
    private SqlConnection _conn = null!;

    [SetUp]
    public async Task Setup()
    {
        _conn = new SqlConnection(TestConfig.DbConnection);
        await _conn.OpenAsync();
    }

    [TearDown]
    public async Task Teardown() => await _conn.DisposeAsync();

    [Test]
    public void Connection_ShouldBeOpen()
    {
        Assert.That(_conn.State, Is.EqualTo(System.Data.ConnectionState.Open));
    }

    [Test]
    public async Task Database_FinanceSmart_ShouldExist()
    {
        await using var cmd = new SqlCommand(
            "SELECT COUNT(*) FROM sys.databases WHERE name = 'FINANCE_SMART'", _conn);
        var count = (int)(await cmd.ExecuteScalarAsync())!;
        Assert.That(count, Is.EqualTo(1));
    }

    [Test]
    public async Task Database_ShouldHaveTables()
    {
        await using var cmd = new SqlCommand(
            "SELECT COUNT(*) FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE'", _conn);
        var count = (int)(await cmd.ExecuteScalarAsync())!;
        Assert.That(count, Is.GreaterThan(0));
    }

    [Test]
    public async Task Trigger_GerenciaSaldo_ShouldExist()
    {
        // O arquivo do repositório se chama TR_GERENCIA_SALDO.sql
        await using var cmd = new SqlCommand(
            "SELECT COUNT(*) FROM sys.triggers WHERE name LIKE '%SALDO%'", _conn);
        var count = (int)(await cmd.ExecuteScalarAsync())!;
        Assert.That(count, Is.GreaterThan(0));
    }
}
```

Como cada parte funciona

Elemento	Explicação e motivo
[TestFixture]	Marca a classe como um conjunto de testes do NUnit.
[Category("Database")]	Etiqueta que permite rodar só estes testes: dotnet test --filter "Category=Database". Útil para rodar o mais rápido primeiro.
[SetUp]	Executa antes de cada teste. Abre uma conexão nova, de modo que cada teste começa do zero e um não interfere no outro.

Elemento	Explicação e motivo
[TearDown]	Executa depois de cada teste e fecha a conexão, liberando recursos mesmo se o teste falhar.
Connection_ShouldBeOpen	Verifica o essencial: o sistema consegue falar com o SQL Server. Se falhar, os demais testes não fazem sentido.
Database_FinanceSmart_ShouldExist	Consulta sys.databases. Confirma que o container db-init criou o banco FINANCE_SMART.
Database_ShouldHaveTables	Consulta INFORMATION_SCHEMA.TABLES. Confirma que o script de criação das tabelas foi executado.
Trigger_GerenciaSaldo_ShouldExist	Consulta sys.triggers. A regra de saldo do sistema depende desta trigger; sem ela, o saldo ficaria inconsistente.
ExecuteScalarAsync	Executa uma consulta e devolve um único valor (aqui, uma contagem). É o jeito mais simples de verificar a existência de algo.
Assert.That(...)	A verificação do teste. Se a condição for falsa, o teste é marcado como falho com uma mensagem clara.
async / await	Operações de rede (banco, HTTP) são assíncronas; await espera o resultado sem bloquear a thread.

Por que consultar o próprio banco? Os testes de interface só mostram que algo deu errado; este teste aponta a *causa* (o banco, a tabela ou a trigger). Em caso de falha, o diagnóstico é imediato.

8. Ambiente de e-mail para testes (Mailpit)

8.1 Por que não usar o Gmail nos testes

O Finance Smart envia e-mails por SMTP (o exemplo de configuração usa o Gmail). Testar com um servidor real tem problemas: é lento, depende de internet, pode bloquear a conta por excesso de envios e exige senhas reais no projeto. A solução padrão é usar um **servidor SMTP falso**.

O **Mailpit** é um servidor SMTP de desenvolvimento: ele recebe os e-mails, não os entrega a ninguém e os mostra em uma página web (porta 8025) e em uma **API**. Assim o teste consegue verificar se o e-mail foi realmente enviado.

8.2 Arquivo docker-compose.test.yml

Fica na raiz do projeto, ao lado do `docker-compose.yml`:

```
services:
  mailpit:
    image: axllent/mailpit
    container_name: mailpit
    environment:
      - MP_SMTP_AUTH_ACCEPT_ANY=1
      - MP_SMTP_AUTH_ALLOW_INSECURE=1
    ports:
      - "8025:8025"   # interface web e API
      - "1025:1025"   # SMTP
```

- **Arquivo separado:** a configuração de teste não altera o compose original; ela é combinada somente quando necessário.
- **MP_SMTP_AUTH_ACCEPT_ANY e ALLOW_INSECURE:** permitem que o email_service se autentique com qualquer usuário e senha, sem TLS, o que simplifica o ambiente de teste.

- **Portas:** 1025 recebe os e-mails (SMTP); 8025 oferece a interface e a API de consulta.

8.3 Configuração do email_service (backend/.env)

O serviço de e-mail lê o arquivo `backend\.env` (definido em `env_file` no compose). Para os testes, ele deve apontar para o Mailpit:

```
EMAIL_HOST=mailpit
EMAIL_PORT=1025
EMAIL_USER=teste@financesmart.local
EMAIL_PASS=qualquercoisa
```

O nome `mailpit` funciona como endereço porque os containers do Docker Compose se enxergam pelo nome do serviço.

8.4 Subindo o ambiente de teste

```
docker compose -f docker-compose.yml -f docker-compose.test.yml up -d --build --force-recreate email_service mailpit
```

O `-f` duas vezes combina os dois arquivos. O `--force-recreate` garante que o email_service releia o .env. Os e-mails aparecem em `http://localhost:8025`.

> **Lembrete:** para voltar a enviar e-mails reais, restaure no `backend\.env` os valores originais do provedor (host, porta, usuário e senha de aplicativo) e recrie o email_service.

9. Teste de recuperação de senha (ForgotPasswordTests.cs)

```
using System.Text.Json;
using Microsoft.Playwright;
using Microsoft.Playwright.NUnit;

namespace FinanceSmart.Tests.Tests;

[TestFixture]
[Category("Email")]
public class ForgotPasswordTests : PageTest
{
    // Use o e-mail de um usuario que EXISTA no sistema
    private const string UserEmail = "usuario.teste@financesmart.local";

    private IAPIRequestContext _mailpit = null!;

    [SetUp]
    public async Task Setup()
    {
        _mailpit = await Playwright.APIRequest.NewContextAsync(
            new() { BaseURL = TestConfig.MailpitUrl });
        await _mailpit.DeleteAsync("/api/v1/messages"); // caixa limpa antes de cada teste
    }

    [TearDown]
    public async Task Teardown() => await _mailpit.DisposeAsync();

    [Test]
    public async Task ForgotPassword_ShouldShowConfirmation_AndSendEmail()
    {
        await Page.GotoAsync(TestConfig.FrontUrl);

        // Seletores: ajuste conforme a tela (use o codegen)
        await Page.GetByText("Esqueci minha senha").ClickAsync();
        await Page.GetByLabel("E-mail").FillAsync(UserEmail);
        await Page.GetByRole(AriaRole.Button, new() { Name = "Enviar" }).ClickAsync();

        // 1) A tela deve dar retorno ao usuario
        await Expect(Page.GetByText("e-mail", new() { Exact = false })).ToBeVisibleAsync();

        // 2) O e-mail deve ter chegado na caixa do Mailpit
```

```
        var message = await WaitForEmailAsync(UserEmail, TimeSpan.FromSeconds(20));
        Assert.That(message, Is.Not.Null, "Nenhum e-mail de recuperacao foi recebido.");
    }

    [Test]
    public async Task ForgotPassword_ShouldCallAuthApi_WithEmail()
    {
        await Page.GotoAsync(TestConfig.FrontUrl);
        await Page.GetByText("Esqueci minha senha").ClickAsync();
        await Page.GetByLabel("E-mail").FillAsync(UserEmail);

        // Captura a chamada do front para o auth_service (porta 8001)
        var response = await Page.RunAndWaitForResponseAsync(
            async () => await Page.GetByRole(AriaRole.Button, new() { Name = "Enviar" }).ClickAsync(),
            r => r.Url.StartsWith(TestConfig.AuthUrl) && r.Request.Method == "POST");

        Assert.That(response.Ok, Is.True, $"API retornou {response.Status}");
    }

    // Consulta a API do Mailpit ate o e-mail chegar (ou estourar o tempo)
    private async Task<JsonElement?> WaitForEmailAsync(string toEmail, TimeSpan timeout)
    {
        var deadline = DateTime.UtcNow + timeout;
        while (DateTime.UtcNow < deadline)
        {
            var res = await _mailpit.GetAsync($"/api/v1/search?query=to:{toEmail}");
            using var doc = JsonDocument.Parse(await res.TextAsync());
            var messages = doc.RootElement.GetProperty("messages");
            if (messages.GetArrayLength() > 0)
                return messages[0].Clone();

            await Task.Delay(1000);
        }
        return null;
    }
}
```

Como cada parte funciona

Elemento	Explicação e motivo
class ... : PageTest	Herdar de PageTest faz o Playwright abrir e fechar um navegador novo para cada teste e disponibiliza a propriedade Page (a aba do navegador) e o método Expect.
[Category("Email")]	Permite rodar só este grupo: dotnet test --filter "Category=Email".
IAPIRequestContext _mailpit	Cliente HTTP do Playwright apontado para a API do Mailpit. Reaproveitamos o Playwright em vez de adicionar outra biblioteca.
DeleteAsync("/api/v1/messages")	Apaga todos os e-mails antes de cada teste. Garante que um e-mail antigo não faça o teste passar por engano.
Page.GotoAsync	Abre o front-end no navegador.
GetByText / GetByLabel / GetByRole	Localizadores recomendados pelo Playwright: encontram elementos pelo que o usuário vê (texto, rótulo, tipo de botão), o que torna o teste mais estável que seletores de CSS.
Expect(...).ToBeVisibleAsync()	Verificação com espera automática: o Playwright tenta por alguns segundos até o elemento aparecer, sem precisar de pausas manuais (sleep).
RunAndWaitForResponseAsync	Executa um clique e, ao mesmo tempo, espera a resposta de rede que atenda à condição. Valida que o front chamou o auth_service (POST) e que a API respondeu com sucesso.
WaitForEmailAsync	Faz polling: consulta o Mailpit a cada segundo, por até 20 segundos, até o e-mail chegar. O envio é assíncrono, então não se deve assumir que o e-mail já chegou no instante do clique.

Elemento	Explicação e motivo
search?query=to:...	Busca na API do Mailpit pelas mensagens destinadas ao e-mail informado.
messages[0].Clone()	Copia o elemento JSON antes de liberar o documento (using), evitando acesso a memória já descartada.

O que cada teste prova

Teste	Prova que
ForgotPassword_ShouldShowConfirmation_AndSendEmail	O usuário consegue pedir a recuperação pela tela, vê uma confirmação e o e-mail realmente é entregue ao servidor SMTP (fluxo de ponta a ponta).
ForgotPassword_ShouldCallAuthApi_WithEmail	O front-end chama corretamente o auth_service e a API responde com sucesso. Em caso de falha, indica se o problema está na API ou no envio do e-mail.

> **Pré-condição:** o e-mail usado em `UserEmail` precisa estar cadastrado no sistema. Se o usuário não existir, o auth_service pode não enviar o e-mail e o teste falhará. Os textos dos seletores ("Esqueci minha senha", "E-mail", "Enviar") devem ser iguais aos da tela real; use o codegen para conferir.

10. Como executar os testes

```
dotnet test                                  # todos os testes
dotnet test --filter "Category=Database"     # somente banco de dados
dotnet test --filter "Category=Email"        # somente recuperação de senha
```

Para ver o navegador abrindo durante o teste (modo visível), útil para depuração:

```
# PowerShell (Windows)
$env:HEADED="1"; dotnet test --filter "Category=Email"

# Linux / Mac
HEADED=1 dotnet test --filter "Category=Email"
```

Ordem recomendada: (1) subir o Docker; (2) rodar o teste de banco, que é o mais rápido e mostra se o ambiente está saudável; (3) subir o Mailpit e rodar o teste de e-mail. Os testes devem ser executados dentro da pasta que contém o arquivo .csproj.

11. Problemas comuns e soluções

Sintoma	Causa provável	Solução
'pwsh' não é reconhecido	PowerShell 7 não instalado	Usar powershell -ExecutionPolicy Bypass -File ... ou instalar com winget install Microsoft.PowerShell.
MSB1003: especifique um arquivo de projeto	Comando dotnet executado fora da pasta do .csproj	Entrar na pasta FinanceSmart.Tests (conferir com dir) antes de rodar.
playwright.ps1 não encontrado	Projeto não compilado ou nome da pasta (net8.0/net11.0) errado	Rodar dotnet build e conferir o nome em bin\Debug.
Container do SQL Server em loop pedindo EULA	Variável ACCEPT_EULA=Y ausente em algum container que usa a imagem do SQL Server	Incluir ACCEPT_EULA=Y no serviço, depois docker compose down -v e subir novamente.

Sintoma	Causa provável	Solução
Teste de banco não conecta	Docker parado, porta 1434 incorreta ou banco ainda iniciando	Verificar docker compose ps e a string de conexão em TestConfig.
E-mail não chega no Mailpit	.env ainda aponta para o Gmail ou o email_service não foi recriado	Conferir EMAIL_HOST=mailpit e rodar --force-recreate email_service.

12. Boas práticas e próximos passos

- **Segredos fora do código:** o docker-compose.yml contém a senha do SQL Server e a SECRET_KEY do JWT em texto aberto. O ideal é movê-las para variáveis de ambiente (${SECRET_KEY}) e trocar a chave atual, já que o repositório é público.
- **Page Object Model:** concentrar os seletores de cada tela em uma classe própria, para que uma mudança na interface exija alteração em um só lugar.
- **Mais cenários:** login, cadastro, criação de receita e despesa e verificação do saldo (testando também a trigger de saldo por meio de uma transação).
- **Testes de API autenticados:** obter o token JWT no auth_service e usá-lo nas chamadas ao transacoes_service.
- **Integração contínua (CI):** executar docker compose e dotnet test automaticamente a cada commit (por exemplo, com GitHub Actions).
- **Relatórios e evidências:** habilitar capturas de tela e traces do Playwright para facilitar a análise de falhas.

13. Glossário

Termo	Significado
E2E (ponta a ponta)	Teste que percorre o sistema como um usuário real, da interface até o banco.
Microserviços	Arquitetura em que o sistema é dividido em serviços pequenos e independentes.
FastAPI	Framework Python para criar APIs web rápidas, com documentação automática (Swagger).
JWT	Token assinado usado para provar que o usuário está autenticado.
SMTP	Protocolo usado para enviar e-mails.
Mailpit	Servidor SMTP falso para desenvolvimento, que guarda os e-mails para consulta.
Trigger	Rotina do banco de dados executada automaticamente quando dados são inseridos ou alterados.
NUnit	Framework de testes para .NET.
Seletor / Localizador	Forma de indicar ao Playwright qual elemento da tela usar.
Polling	Consultar repetidamente até que uma condição seja atendida ou o tempo se esgote.
Assert	Verificação dentro do teste; se for falsa, o teste falha.